Complexity and Big-O Notation

- **Definition of Complexity**:
  - Complexity of a function describes the relationship between the size of the input and the difficulty of running the function.
  - The input size is often denoted as *n* and can represent tangible factors like the length of an array.

- **Basic Operations**:
  - Operations such as additions, multiplications, and function calls are considered to assess complexity.
  - The number of basic operations is used as an approximation of a function's run time.

- **Big-O Notation**:
  - A common way to express the growth rate of an algorithm as input size increases.
  - It only considers the highest power term and ignores constants to focus on large-scale behavior.
  - Example: A function requiring 4n^2 + n + 1 basic operations is simplified to O(n^2).

Examples of Algorithm Complexity

- **Quadratic Complexity O(n^2):**
- Example: A nested loop structure where the total number of operations grows proportionally to n^2, such as:

```python
def f(n):
    out = 0
    for i in range(n):
        for j in range(n):
            out += i * j
    return out
```

- Total operations: n^2, leading to a complexity of O(n^2).

- **Linear Complexity O(n):**
- Example: An iterative Fibonacci function where the number of operations grows linearly with input size:

```python
def my_fib_iter(n):
    out = [1, 1]
    for i in range(2, n):
        out.append(out[i - 1] + out[i - 2])
    return out
```

- This function grows with O(n), as the for-loop runs n times.

Recursive Complexity and Exponential Time

- **Exponential Complexity O(2^n):**
  - Example: Recursive Fibonacci function where each call leads to two more recursive calls:

```python
def my_fib_rec(n):
    if n < 2:
        return 1
    else:
        return my_fib_rec(n-1) + my_fib_rec(n-2)
```

  - The number of recursive calls doubles with each increment in n, leading to an exponential growth rate of O(2^n).

- **Logarithmic Complexity O(log n):**
  - Example: A function that divides the input in half each time:

```python
def my_divide_by_two(n):
    out = 0
    while n > 1:
        n /= 2
        out += 1
    return out
```

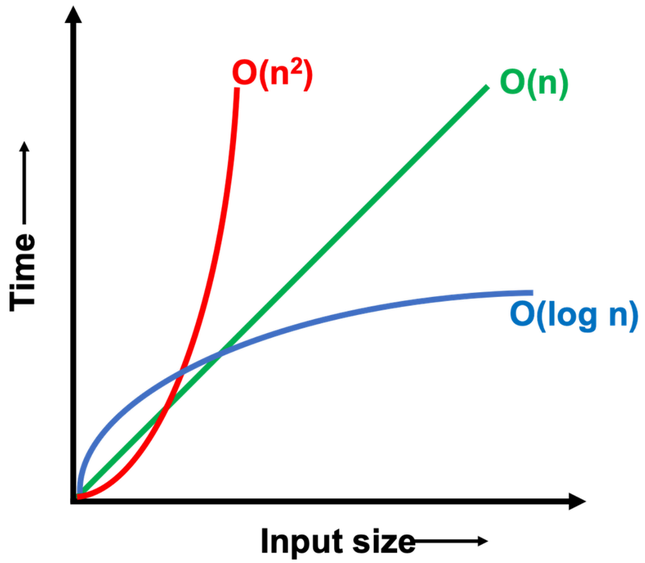
  - The complexity is O(log n) since the input size is halved during each iteration.

Why Complexity Matters

- **Complexity** measures how an algorithm's runtime or space requirements grow with input size.

- **Different types of complexity**:
  - Logarithmic: O(log n)
  - Linear: O(n)
  - Quadratic: O(n^2)
  - Exponential: O(2^n)

- **Impact of complexity on performance**:
  - **Exponential growth**: Adding resources barely increases the size of solvable problems.
  - **Polynomial time**: More manageable as problem size increases.
  - **Logarithmic algorithms**: Can handle significantly larger problems as resources scale.



Visual Representation:

- The relationship between input size and computation time (using logarithmic, linear, quadratic, and exponential growth) is shown through this graph.

Algorithm Scaling and Performance

Scaling Example: Exponential vs Polynomial Time

- **Exponential Algorithm (e.g., O(2^n)):**
  - Doubling computational power increases the solvable problem size by only one unit.
  - For large inputs, exponential algorithms quickly become infeasible.
- **Polynomial Algorithm (e.g., O(n^c)):**
  - Doubling resources can increase the solvable problem size by a factor of $\sqrt[c]{2}$.
  - More efficient for larger problems.

Real-world Impact:

- **Fibonacci Example:**
  - Recursive solution (exponential time) vs iterative solution (polynomial time).
  - In practice, polynomial algorithms are chosen for performance-critical applications.

NP-Complete Problems and Engineering

NP-Complete Problems:

- **Definition**: Problems for which no polynomial time algorithm is known.

- **Examples**:
  - **Traveling Salesman Problem (TSP)**
  - **Set Cover Problem**

- **Significance**:
  - These problems have significant applications in **logistics** and **operations research**.
  - They are crucial in fields like **encryption**, where the difficulty of solving NP-Complete problems secures data (e.g., in banking systems).

Profiling in Python

- **What is Profiling?**

Profiling in Python involves measuring the time and resources used by code to identify bottlenecks and optimize performance.

- **Purpose of Profiling:**
  - Helps developers focus on improving parts of code that consume most time or resources.
  - Useful in optimizing the performance of numerical and computational programs.

- **Common Python Profiling Tools:**
  - %time and %timeit: Magic commands used in Jupyter notebooks to check execution time of code.
  - Python’s built-in cProfile: Provides detailed reports on function calls and time spent.

Magic Commands for Time Measurement

- **Single Line Timing:**
  - %time measures the execution time of a single line of code.
  - Example: %time sum(range(200))

Output: CPU times: user 6 μs, sys: 1 μs, total: 7 μs | Wall time: 9.06 μs

- **Code Block Timing:**
  - %%time measures the execution time of all code within a Jupyter cell.
  - Example:

```python
%%time
s = 0
for i in range(200):
    s += i
```

- **Repetitive Timing:**
  - %timeit: Runs code multiple times and provides an average time.
  - Example: %timeit sum(range(200))

Output: 1.24 μs ± 70.6 ns per loop (1000000 loops each)

Advanced Profiling with Line Profiler

- **What is Line Profiling?**
  - Used to measure the time taken by individual lines of code.
  - Helps identify specific bottlenecks within a function.

- **Example: Using** line_profiler**:**
  - After installing the line_profiler package, use %lprun to profile specific functions.
  - Code Example:

```python
%load_ext line_profiler
%lprun -f slow_sum slow_sum(1000, 10000)
```

```
Timer unit: 1e-06 s

Total time: 6.1411 s
File: <ipython-input-20-cc5de53096ac>
Function: slow_sum at line 1

Line #      Hits         Time  Per Hit   % Time  Line Contents
==============================================================
     1                                           def slow_sum(n, m):
     2
     3      1001        301.0      0.3      0.0      for i in range(n):
     4                                                   # we create a size m array of random numbers
     5      1000      87876.0     87.9      1.4          a = np.random.rand(m)
     6
     7      1000        439.0      0.4      0.0          s = 0
     8                                                   # in this loop we iterate through the array
     9                                                   # and add elements to the sum one by one
    10  10001000    2463579.0      0.2     40.1          for j in range(m):
    11  10000000    3588901.0      0.4     58.4              s += a[j]
```

- **Output Analysis:**
  - Line-by-line summary of execution time.
  - Shows that lines 10 and 11 are responsible for the majority of the runtime (adding elements one by one to a sum).